Advanced R for Econometricians (Summer 2026)

Advanced R Concepts — Notes and Examples

Martin Arnold, Martin Schmelzer

Make sure the `lobstr` package is installed and attached.

```
install.packages("lobstr")
library(lobstr)
```

Assignment Operators

- R interprets `a <- b = 1` as `'<-<-'(a, b = 1, value = 1)`, which errors.
- `<<-` walks up enclosing environments until it finds an existing binding; if none exists, it creates one in the global environment.

Bindings

- Names are bindings (references) to values in memory; they do not "contain" values.
- `x <- c(1, 2, 3); y <- x` creates two bindings to the same value, with the same memory address.
- This matters for performance: passing large objects does not copy by default.

Copy-on-modify

- Copy-on-modify protects against accidental side effects, but it can be expensive.
- Modifying one of multiple bindings triggers a copy; after separation, objects can diverge safely.
- Performance hinges on how many bindings point to the same value.

Copy-on-modify: Tracking with `tracemem()`

- `tracemem()` reports old/new addresses and, in function calls, includes the call stack that triggered the copy.
- After a first copying modification, later modifications can be in-place once an object has a single binding.
- `untracemem()` should be used to stop tracing once diagnostics are complete.

Copy-on-modify: Function Calls

Solution to the task

- If `f <- function(a) { a[[1]] <- 0; a }`, then calling `f(x)` triggers one copy because `a` is modified.
- `z <- f(x)` does not add another copy on top of that modification copy; `z` binds to the modified object returned from `f()`.
- For an identity function `f <- function(a) a`, `f(x)` and `z <- f(x)` make no copy; they only create/add bindings.

Copy-on-modify: Lists and Data Frames

Solution to the task

- List modification `l2[[3]] <- 4` performs a shallow copy: the list spine is copied, elements can remain shared references.
- For a data frame `df`, `df[1, ] <- c(42, 42)` is expensive because row replacement touches all columns, so multiple vectors are copied.
- `df$x <- c(7, 7, 7)` only replaces one column vector, so less copying is required.
- `ref(l1, l2)` after list modification reveals shared element references, confirming shallow-copy behavior.

Copy-on-modify: Exercises

Solutions

1. `tracemem(1:10)` is not useful because `1:10` is not a binding to a named object.
2. `x <- c(1L, 2L, 3L); x[[3]] <- 4` copies because `x` is integer and `4` is double, so coercion to double is required.
3. `obj_size(1:10)` and `obj_size(1:1e6)` are similar because `:` stores compact integer sequences using start/end metadata.
4. Rewriting the loop with `lapply()` constructs new outputs instead of mutating a shared object in-place, so it avoids repeated data-frame replacement copies.

Modify-in-place

- In-place modification occurs when an object has one binding.
- Environments are always modified in-place.
- In RStudio, the Environment pane can create an extra binding, so in-place demos may appear to copy.
- For clean in-place tests, prefer the plain R console (no extra IDE-held binding).

Case Study: Copy-on-Modify Inferno (Concept)

- Iterative replacement on a data frame (`x[[i]] <- ...`) can trigger several copies per iteration because `[[<-.data.frame` uses temporary bindings and rebuilds structure.
- Converting to list first (`y <- as.list(x)`) can reduce this to one initial list-spine copy, after which element updates are effectively in-place.
- This motivates data-structure-aware optimization before moving to C++/`Rcpp`.

Garbage Collection

- R uses tracing garbage collection: unreachable objects are freed automatically.
- Rebinding (`x <- 2:4`) or removing (`rm(x)`) can make old values unreachable.
- `gc()` is mostly diagnostic (prints memory summary): `vcells` for vector memory, `ncells` for non-vector objects.
- `gcinfo(TRUE)` prints a message whenever GC runs, useful when diagnosing memory pressure.

Regular Functions vs. Primitives

- Regular functions (`typeof == "closure"`) have formals and a body.
- Primitives (`sum`, `+`, `[`) are implemented in C and have no R-level formals/body.
- Closures "carry" their defining environment; primitives bypass most standard R call machinery.

First-Class Functions

Solution to the task

- `(` is a primitive that returns its argument (semantically like `function(x) x`).
- `(x <- 5)` is valid: it performs the assignment and visibly returns/prints the assigned value.
- Anonymous functions can be stored, passed, and called immediately (including modern `\(x)` lambda syntax).

R's Environment Hierarchy

- Function execution environments are created per call; their parent is the function's defining environment.
- Package environments form the search path between `.GlobalEnv` and `base`.
- Lookup that reaches `R_EmptyEnv` without finding a name throws "object not found".
- `environment(f)` gives the enclosing environment used as parent of `f()` execution environments.
- `search()` reveals package lookup order; load order can affect masking.

Lexical Scoping

- Name masking: inner names mask outer names.
- Functions before variables: in a call, R ignores non-function objects with that name.
- Execution environments: each call gets an ephemeral environment.
- Dynamic lookup: free variables are resolved when the function runs.
- In a call like `z(x + y)`, R looks for a function named `z` (ignoring non-function `z` values in that call position).

Lazy Evaluation

- Arguments are promises and are evaluated only when needed.
- A promise is forced at most once; then its value is cached.
- Default arguments are evaluated in the function environment at call time.
- Unevaluated arguments are never forced; e.g., `ignore(x = stop("never evaluated"))` is safe if `x` is unused.

Lazy Evaluation: Default Arguments

- Defaults can reference names created inside the function body, because they are resolved at call time in the function environment.
- This behavior is often useful for concise dependent defaults, but can hide errors if names are misspelled.

Lazy Evaluation: Short-circuit Evaluation

Solution to the task

1. `x > 0` is problematic for `x <- NULL` because it yields `logical(0)`, which `if` cannot use.
2. No error occurs because `&&` short-circuits; once `!is.null(x)` is `FALSE`, the right-hand side is not evaluated.

Function Forms

- All calls can be written in prefix form, including operators and special forms.
- Custom infix operators use `%...%`.
- Replacement functions (e.g. `name<-`) must return the full modified object.
- Prefix rewrites like `` `+`(1, 2) `` and `` `[`(1:3, 3) `` are useful for understanding parser behavior.

Function Forms: Infix Operators

- User-defined infix operators are ordinary functions wrapped with `%` delimiters (for example, `%+%`, `%between%`).
- They improve readability in domain-specific expressions.

Function Forms: Replacement Functions

- Replacement functions must accept `(x, value)` and return the full modified `x`.
- Replacement syntax desugars to a function call with rebinding and temporary objects, so copying behavior mirrors copy-on-modify semantics.

Function Forms: Exercises

Solutions

1. `f(x = z^2, z = 2)` fails because `x = z^2` is resolved in the calling environment where `z` is missing.

   Fix with `with()`:

```r
f <- function(x, z) z + x^2
with(list(z = 2), f(x = z^2, z = 2))
```

2. `...` captures additional arguments (named or unnamed), preserves names, and can be forwarded to other functions.

3. Replacement function for winsorisation:

```r
`winsorise<-` <- function(x, value) {
  stopifnot(is.numeric(value), length(value) == 2)
  lower <- value[[1]]
  upper <- value[[2]]
  x[x < lower] <- lower
  x[x > upper] <- upper
  x
}

returns <- c(-0.52, 0.03, 0.15, 1.20, -0.08, 0.97)
winsorise(returns) <- c(-0.5, 0.5)
returns
```